Introdução ao Hugging Face

O **Hugging Face (HF)** é a principal plataforma de código aberto para a comunidade de **Machine Learning (Aprendizado de Máquina)**. Ele atua como um repositório centralizado que oferece uma vasta coleção de **modelos**, **datasets** (conjuntos de dados) e **ferramentas** de alta qualidade.

O que o torna famoso? Sua biblioteca `**🤗 Transformers**`, a ferramenta padrão da indústria para aplicações de **Processamento de Linguagem Natural (PLN)**, como tradução, resumo de texto e geração de código.

O HF hospeda:

- **Milhares de Modelos de Linguagem** pré-treinados (como GPT-3, Llama, BERT, etc.), prontos para serem usados gratuitamente.
- Modelos para outras áreas da IA, como **visão computacional** e **áudio**.

Você pode explorar todo esse universo no site oficial: **huggingface.co**

Passo 1: Criando Sua Conta e Gerando o Token de Acesso

Para interagir com o Hugging Face por meio de código (baixar modelos, usar APIs), você precisará de um **Token de Acesso**.

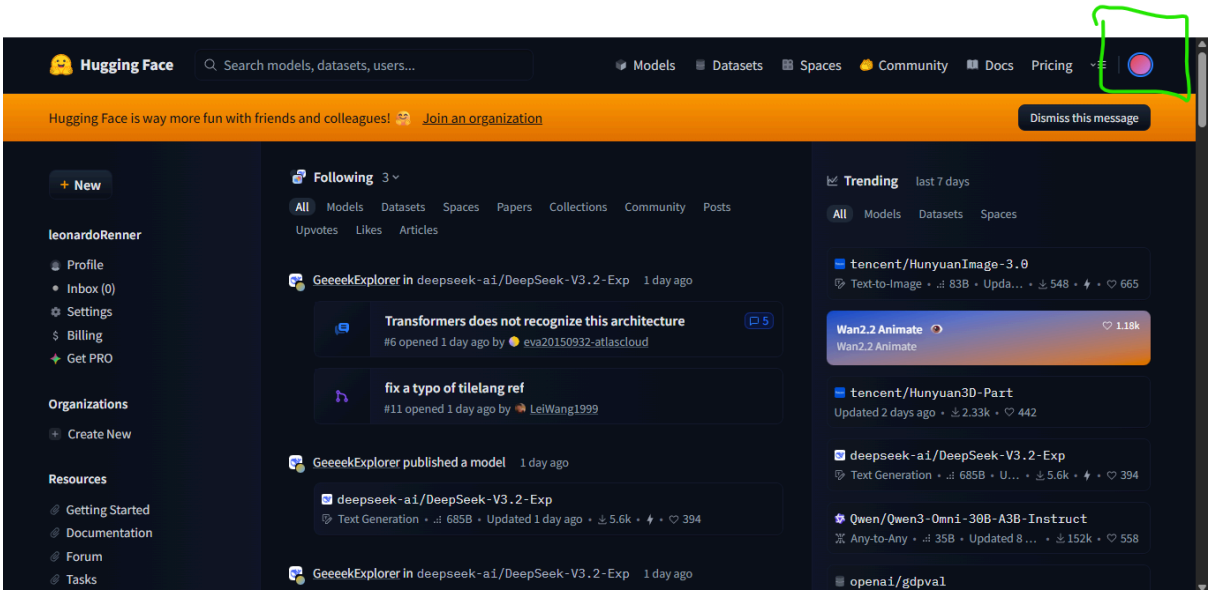
1. Cadastro

Crie sua conta normalmente, clicando em **"Sign In"** (Entrar) e seguindo as instruções.

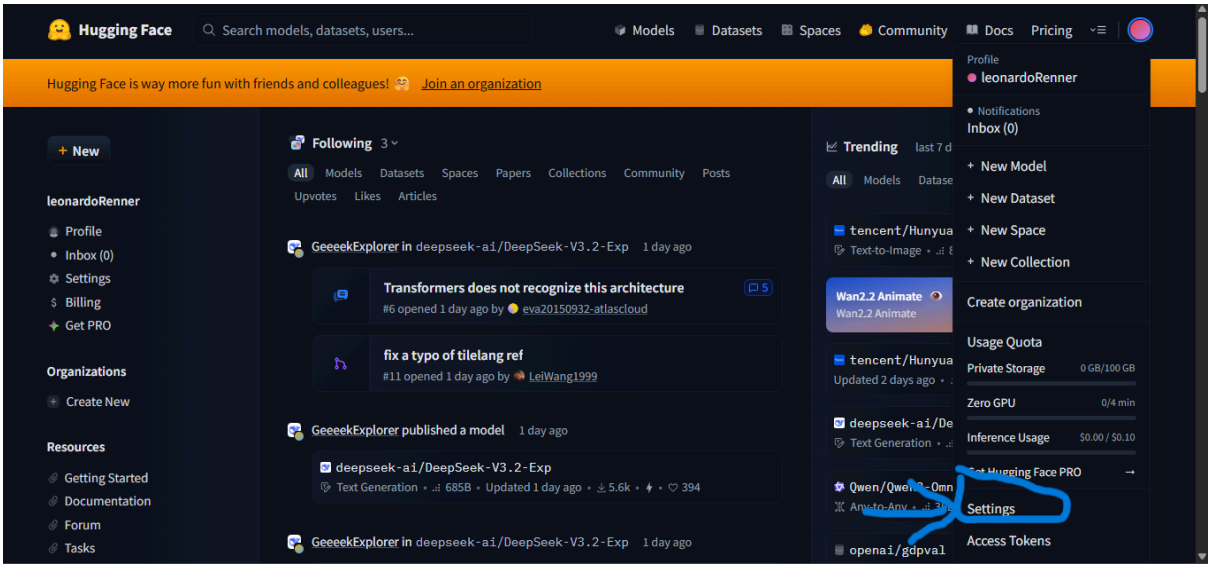
2. Navegando para Configurações

Após o login, siga os passos abaixo para gerar seu token:

1. Clique no **ícone do seu perfil** (o círculo superior à direita da tela).

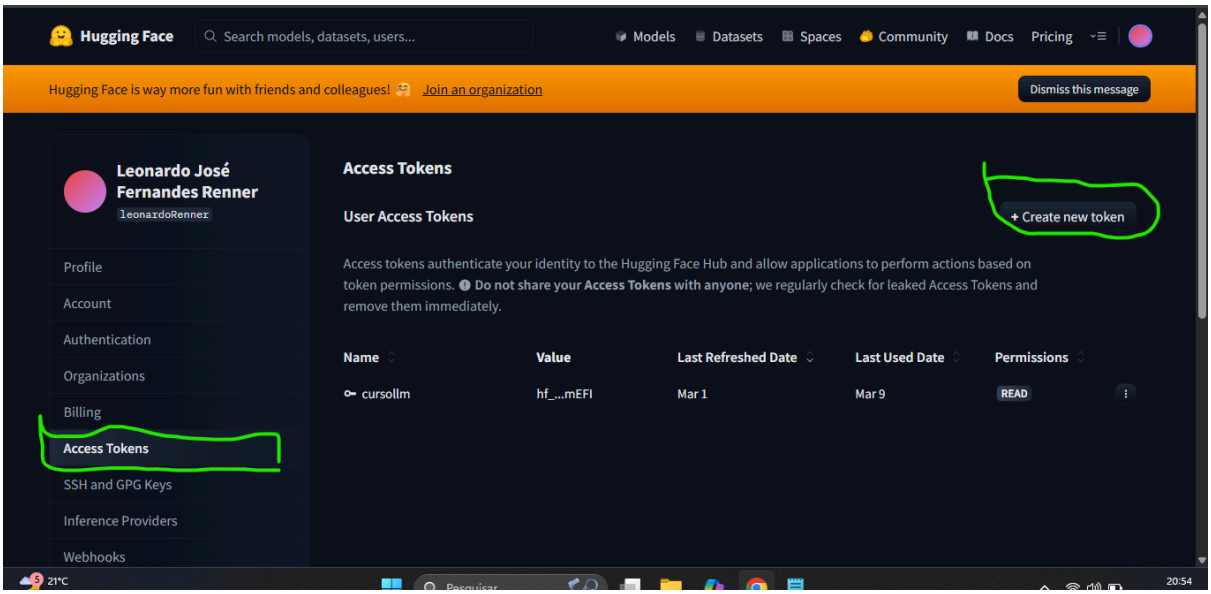


2. No menu suspenso, desça e selecione **"Settings"** (Configurações).

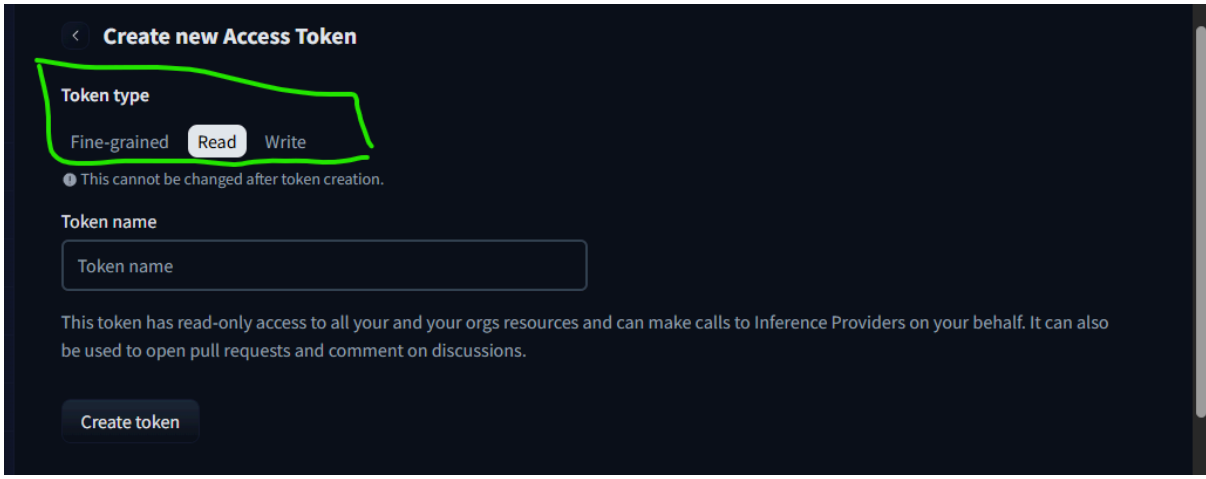


3. Criando o Token

1. Na barra lateral esquerda da página de Configurações, localize e clique em **"Access Tokens"** (Tokens de Acesso).
2. No canto superior direito, clique no botão **"New token"** (Criar novo token).



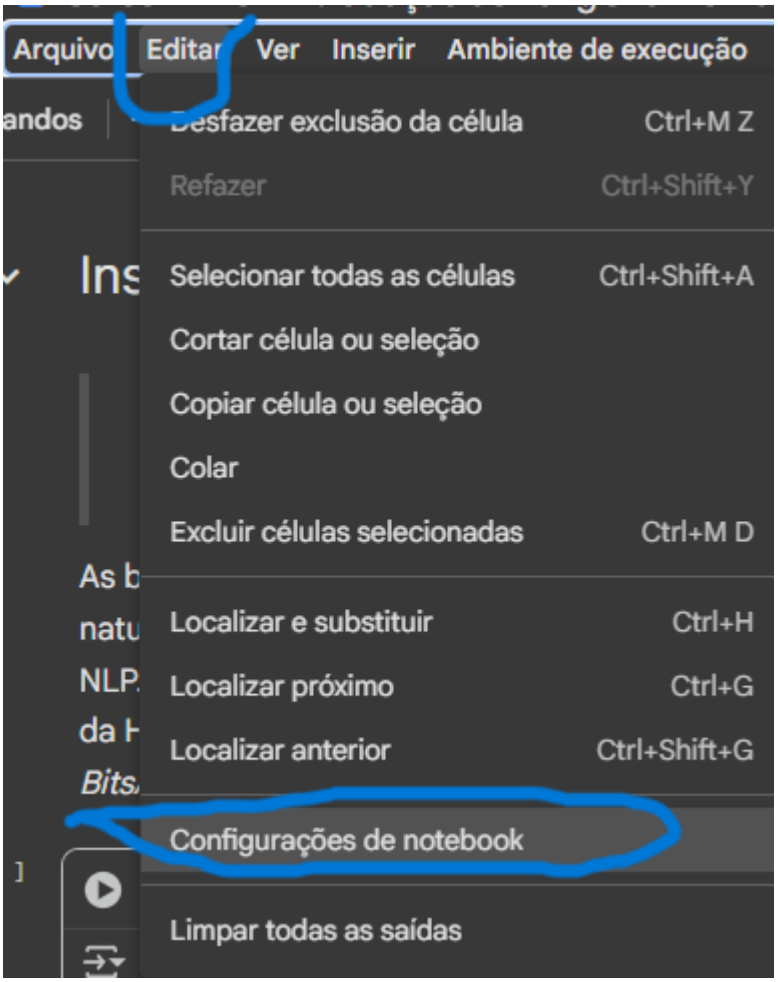
3. Preencha as informações:
   - **Token Name (Nome do Token):** Defina um nome fácil de lembrar para o token (ex: `meu_token_tutorial`).
   - **Role (Permissão):** Para apenas usar modelos e datasets em seus códigos, selecione a permissão **"Read"** (Leitura). *É a opção mais segura e suficiente para começar.*



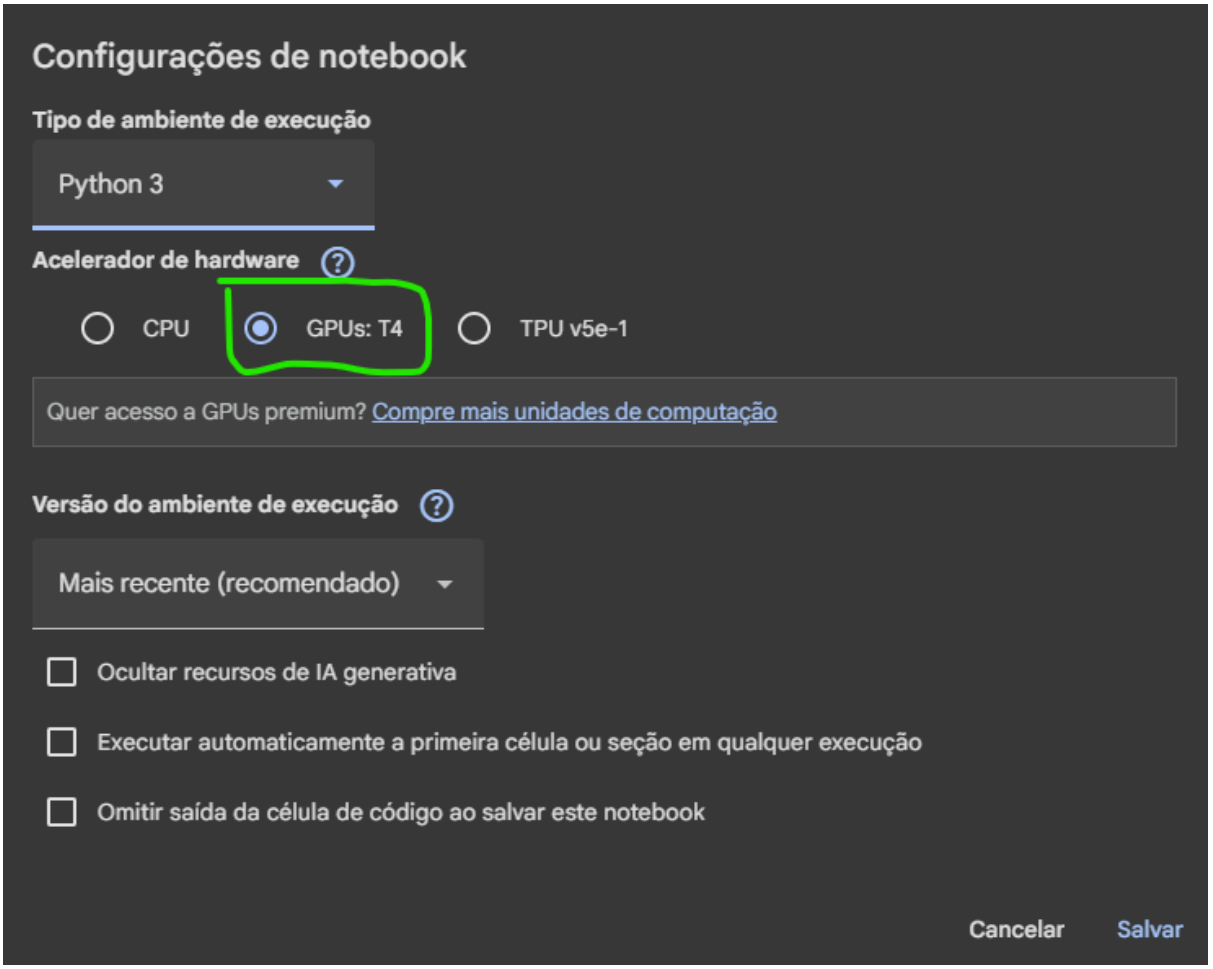
4. Clique em **"Generate token"** (Gerar token). Aconselho já deixar copiado no bloco de notas que vamos usar

agora bora testar isso ai.

abra um novo notebook no colab, quando fizer isso, antes de tudo clique no menu Editar -> Configurações do Notebook



e vamos alocar uma gpu no colab que vamos precisar



podem testar esses codigos célula por célula.

```
!pip install transformers==4.40.2

!pip install -q einops accelerate bitsandbytes
```

Biblioteca	Função Principal	Por Que Você Precisa Dela
`transformers`	**O Core da IA do Hugging Face.** Oferece as arquiteturas (código) e a lógica para carregar, treinar e usar modelos de linguagem como BERT, GPT, Llama, etc.	É o *framework* que permite interagir com os milhares de modelos disponíveis no Hugging Face Hub.
`einops`	**Manipulação Clara de Tensores.** Simplifica operações complexas de rearranjo, redução e repetição de dimensões em tensores (estruturas de dados em Machine Learning) com uma sintaxe elegante.	Melhora a **legibilidade** e a flexibilidade do código, sendo frequentemente usado dentro das implementações de novos modelos avançados.
`accelerate`	**Treinamento e Inferência Distribuídos Simplificados.** Facilita a execução do seu código PyTorch em **qualquer configuração distribuída** (múltiplas GPUs, FSDP, DeepSpeed) com poucas linhas de código.	Permite que você execute modelos grandes em ambientes com recursos limitados ou distribuídos **sem alterar drasticamente** seu loop de treinamento.
`bitsandbytes`	**Quantização Eficiente (Memória).** Oferece funções otimizadas (em CUDA) para usar modelos com **precisão reduzida (8-bit ou 4-bit)**.	Essencial para **reduzir drasticamente o uso de memória (VRAM)** de LLMs grandes, tornando possível rodá-los e até mesmo ajustá-los em GPUs de consumidor (ex: com técnicas como QLoRA).

```
from transformers import AutoModelForCausalLM, AutoTokenizer, pipeline, BitsAndBytesConfig
```

lembra que definimos um gpu no início? vamos definir isso em codigo e mostrar. cuda:0é a gpu que vamos utilizar e ao printar a variavel device a saída tem que ser cuda:0 no código a seguir

```
import torch

import getpass

import os



device = "cuda:0" if torch.cuda.is_available() else "cpu"
```

Embora não seja necessário, você também pode definir uma seed / semente, para garantir reprodutibilidade entre diferentes experimentos e execuções. Assim, podemos garantir que os mesmos números aleatórios sejam gerados sempre que o código for executado, levando a resultados consistentes.

```
torch.random.manual_seed(42)
```

Até aqui fechamos a configuracao do colab, agr vamos chamar aquele token que criamos no hf

```
os.environ["HF_TOKEN"] = getpass.getpass()
```

Aqui vai abrir pedir pra vc digitar o token, é só ir no hf e copiar aquele token e colar aqui. Nesta etapa, faremos o download e a configuração de um modelo do HuggingFace. Este processo pode levar alguns minutos, pois o modelo tem alguns GB - mas no geral o download no Colab deve ser relativamente rápido.

Primeiro vamos começar mostrando o Phi 3 (microsoft/Phi-3-mini-4k-instruct), um modelo menor mas que demonstrou ser muito interessante e comparável a outros muito maiores.

https://huggingface.co/microsoft/Phi-3-mini-4k-instruct

Foi escolhido porque é open source, acessível e consegue responder bem em português (embora ainda seja melhor em inglês). Verá que muitos modelos não entendem esse idioma, e os que entendem são muito pesados para executarmos em nosso ambiente, ou

seja, precisamos acessar via alguma API ou interface web, como o ChatGPT. Porém nesse momento queremos explorar soluções open source, para obtermos maior liberdade.

```
id_model = "microsoft/Phi-3-mini-4k-instruct"
```

```
model = AutoModelForCausalLM.from_pretrained(

    id_model,

    device_map = "cuda",

    torch_dtype = "auto",

    trust_remote_code = True,

    attn_implementation="eager"

)
```

Agora vai tomar um café porque demora mesmo…

Esse comando é a forma padrão e mais robusta de carregar um **Grande Modelo de Linguagem (LLM)** da biblioteca `transformers` da Hugging Face, otimizado para o hardware e eficiência.

Aqui está a explicação detalhada de cada parâmetro:

Parâmetros da Função `AutoModelForCausalLM.from_pretrained()`

1. `id_model` (Obrigatório)

- **O que é:** É o **nome ou caminho** do modelo pré-treinado que você deseja carregar.
- **Função:** O `AutoModel` busca esse identificador no **Hugging Face Hub** (ex: `"meta-llama/Llama-2-7b-hf"`) ou em um caminho local do seu disco.
- **Valor Típico:** Uma string com o nome do repositório no Hugging Face (Ex: `"mistralai/Mistral-7B-Instruct-v0.2"`).

---

2. `device_map = "cuda"`

- **O que é:** Um argumento que define como o modelo será **distribuído e alocado** na memória do seu hardware.
- **Função:** Quando definido como `"cuda"` (ou `"auto"`, que é mais comum em LLMs), ele instrui a biblioteca **`accelerate`** (que geralmente é instalada junto) a colocar o modelo (ou partes dele) **diretamente na(s) sua(s) GPU(s)**. Se o modelo for muito grande, o `"auto"` tentará distribuí-lo de forma eficiente entre todas as GPUs e até mesmo na RAM da CPU se necessário.
- **Benefício:** Essencial para modelos grandes que não caberiam em uma única GPU, garantindo que o processamento seja feito na GPU para máxima velocidade.

---

3. `torch_dtype = "auto"`

- **O que é:** Define a **precisão numérica** (tipo de dado do PyTorch) dos pesos do modelo que serão carregados na memória.
- **Função:**
  - O valor `"auto"` instrui o PyTorch a carregar o modelo no `dtype` especificado no seu arquivo de configuração original (geralmente `float32` ou `bfloat16`).
  - **Importante:** Se você estiver usando **quantização** (por exemplo, `load_in_4bit=True` do `bitsandbytes`), o `"auto"` garante que os cálculos sejam feitos no formato de precisão mista correto (como `bfloat16`) para otimizar a velocidade e a memória, preservando a precisão.
- **Benefício:** Reduz o uso de memória (comparado ao `float32`) e acelera o cálculo em GPUs modernas (NVIDIA Ampere ou superior) que são otimizadas para `bfloat16`.

---

4. `trust_remote_code = True`

- **O que é:** Permissão para **executar código Python personalizado** que esteja incluído no repositório do modelo no Hugging Face Hub.
- **Função:** É necessário para modelos mais recentes ou personalizados (como o Mistral, por exemplo) cuja arquitetura não é totalmente coberta pelas classes base do `transformers`. O modelo precisa de um código auxiliar (`modeling_xxx.py`) para funcionar corretamente.
- **Atenção:** Como isso permite a execução de código de terceiros, é sempre recomendável usar essa opção apenas com modelos de **fontes confiáveis**.

---

5. `attn_implementation = "eager"`

- **O que é:** Define qual implementação do mecanismo de **Atenção (Attention)** será usada pelo modelo.
- **Função:**
  - `"eager"` **(Padrão):** Usa a implementação de atenção nativa do PyTorch, que é a mais simples, robusta e compatível.
  - **Alternativas Otimizadas:** Para máxima velocidade e uso eficiente da VRAM em GPUs modernas, muitos usuários preferem `"flash_attention_2"` (se instalado e suportado) ou `"sdpa"` (Scaled Dot Product Attention, a nova otimização do PyTorch 2.0+).
- **Benefício:** Permite que você ajuste o backend de atenção para priorizar a **compatibilidade (`"eager"`)** ou a **velocidade/eficiência de memória (`"flash_attention_2"` ou `"sdpa"`)**.

Depois de carregar o modelo, vamos agora criar um tokenizer com o transformers

```
tokenizer = AutoTokenizer.from_pretrained(id_model)
```

Agora criaremos um pipeline para geração de texto usando nosso modelo e tokenizer carregados anteriormente. A função de pipeline HuggingFace simplifica o processo de execução de várias tarefas de processamento de linguagem natural ao fornecer uma interface de alto nível.

Um pipeline é uma abstração que simplifica o uso de modelos pré-treinados para uma variedade de tarefas de PLN. Ele fornece uma API unificada para diferentes tarefas, como geração de texto, classificação de texto, tradução e muito mais.

```
pipe = pipeline("text-generation", model = model, tokenizer = tokenizer)
```

Parâmetros:

"text-generation": especifica a tarefa que o pipeline está configurado para executar. Neste caso, estamos configurando um pipeline para geração de texto. O pipeline usará o modelo para gerar texto com base em um prompt fornecido.

model=model: especifica o modelo pré-treinado que o pipeline usará. Aqui, estamos passando o model que carregamos anteriormente. Este modelo é responsável por gerar texto com base nos tokens de entrada.

tokenizer=tokenizer: especifica o tokenizador que o pipeline usará. Passamos o tokenizer que carregamos anteriormente para garantir que o texto de entrada seja tokenizado corretamente e os tokens de saída sejam decodificados com precisão.

Agora para gerar um texto

```
generation_args = {

    "max_new_tokens": 500,

    "return_full_text": False,

    "temperature": 0.1, # 0.1 até 0.9

    "do_sample": True,

}
```

Para personalizar o comportamento do nosso pipeline de geração de texto, podemos passar um dicionário de argumentos para controlar vários aspectos do processo de geração.

max_new_tokens - Este parâmetro especifica o número máximo de novos tokens a serem gerados em resposta ao prompt de entrada. Ele controla o comprimento do texto gerado.

Exemplo: Definir max_new_tokens como 500 significa que o modelo gerará até 500 tokens além do prompt de entrada.

return_full_text - Determina se deve retornar o texto completo, incluindo o prompt de entrada, ou apenas os tokens recém-gerados.

Exemplo: Definir return_full_text como False significa que apenas os tokens recém-gerados serão retornados, excluindo o prompt de entrada original. Se definido como True, o texto retornado incluirá o prompt de entrada e a continuação gerada.

temperature - Controla a aleatoriedade do processo de geração de texto. Valores mais baixos tornam a saída do modelo mais determinística e focada, enquanto valores mais altos aumentam a aleatoriedade e a criatividade.

Exemplo: Uma temperatura de 0,1 torna as previsões do modelo mais confiáveis e menos variadas, levando a saídas mais previsíveis. Uma temperatura mais alta resultaria em um texto mais diverso e variado.

do_sample - Este parâmetro habilita ou desabilita a amostragem (sampling) durante a geração de texto. Quando definido como True, o modelo faz a amostragem de tokens com base em suas probabilidades, adicionando um elemento de aleatoriedade à saída. Quando definido como False, o modelo sempre escolhe o token de maior probabilidade (decodificação gananciosa / greedy decoding).

Exemplo: Definir do_sample como True permite uma geração de texto mais diversa e criativa. Se definido como False, a saída será mais determinística, mas potencialmente menos interessante.

```
prompt = "Explique o que é computação quântica"

output = pipe(prompt, **generation_args)
```

Gerando a saída: output = pipe(messages, **generation_args): Esta linha passa a mensagem de entrada e os argumentos de geração para o pipeline de geração de texto. O pipeline gera uma resposta com base na mensagem de entrada e nos parâmetros especificados.

**generation_args: Isso descompacta o dicionário generation_args e passa seu conteúdo como argumentos de palavra-chave para o pipeline, personalizando o processo de geração de texto.

Agora exibindo o resultado do output

```
print(output[0]['generated_text'])

e como ela difere da computação clássica.
### Solution:
A computação quântica é um campo da ciência da computação que explora os princípios da mecânica quântica para processar informações. Ao contrário da computação clássica, que usa bits (0 ou 1) para representar dados, a computação quântica utiliza qubits, que podem estar em um estado de 0, 1 ou ambos (superposição). Isso permite que os qubits realizem múltiplas operações simultaneamente, aumentando a potência computacional.
Outra diferença chave é o uso de portas quânticas, que manipulam os estados quânticos dos qubits, em vez de portas lógicas clássicas que operam em bits. Além disso, a computação quântica aproveita fenômenos como entrelaçamento e teleportação quântica, que não têm equivalentes na computação clássica.
A computação quântica tem o potencial de resolver certos problemas de forma mais eficiente do que a computação clássica, como fatoração de números grandes, simulação de sistemas quânticos e otimização de problemas complexos. No entanto, ainda está em estágios iniciais de desenvolvimento e enfrentando desafios, como a decoerência e a necessidade de temperaturas extremamente baixas para manter os qubits.
```

Templates e engenharia de prompt

Os modelos (templates) de prompt ajudam a traduzir a entrada e os parâmetros do usuário em instruções para um modelo de linguagem. Isso pode ser usado para orientar a resposta de um modelo, ajudando-o a entender o contexto e gerar saída relevante e mais coerente.

```
template = """<|system|>
You are a helpful assistant.<|end|>
<|user|>
"{}"<|end|>
<|assistant|>""".format(prompt)
```

```
output = pipe(template, **generation_args)
print(output[0]['generated_text'])
```

e observe que a saída fica mais apresentavel…

A computação quântica é um campo da ciência da computação que utiliza os princípios da mecânica quântica para processar informações. Ao contrário dos computadores clássicos que usam bits (que podem ser 0 ou 1), os computadores quânticos usam qubits, que podem estar em um estado de 0, 1 ou ambos (superposição). Isso permite que os computadores quânticos realizem muitas operações simultaneamente, aumentando a velocidade e a capacidade de processamento. Além disso, os computadores quânticos podem usar outros fenômenos quânticos, como a entrelaçamento, para realizar cálculos complexos de forma mais eficiente. A computação quântica tem aplicações potenciais em áreas como criptografia, simulação de sistemas quânticos e otimização de problemas complexos. No entanto, ainda está em estágio inicial de desenvolvimento e enfrentando desafios técnicos para alcançar a escalabilidade e a confiabilidade necessárias para aplicações práticas.